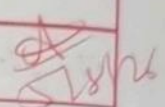
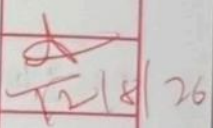
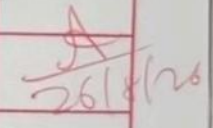
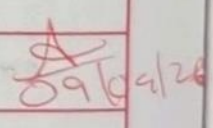
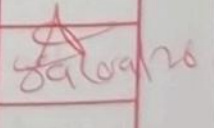




TABLE OF CONTENT

Sr. No.	Experiment Title	Page No		Date of Performance	Date of Submission	Marks	Sign.
		To	From				
01.	plot DFT of a sequence without using the input MATLAB functions	05	01	17/06/26	01/07/26	09	
02	Plot IDFT of a sequence without using the inbuilt MATLAB function	13	07	01/07/26	08/07/26	09	
03	Plot circular convolution of given sequence using MATLAB function.	21	15	08/07/26	05/08/26	08	
04.	Plot FFT on a given sequence and plot the magnitude and phase response of the same. using MATLAB function.	27	22	15/07/26	05/08/26	08	
05.	Plot on a given sequence and plot the magnitude and phase response of the same.	33	29	22/07/26	05/08/26	08	

TABLE OF CONTENT

Sr. No.	Experiment Title	Page No		Date of Performance	Date of Submission	Marks	Sign.
		To	From				
06	To implement low pass FIR filter for given sequence	39	35	29/07/26	05/08/26	09	
07	To implement high pass FIR filter for given sequence.	43	41	05/08/26	12/08/26	09	
08	To implement Band pass FIR filter for given sequence.	47	45	12/08/26	26/08/26	09	
09	To design a Hilbert transform function using Bartlett and Hanning windows.	51	49	26/08/26	02/09/26	08	
10.	To design a Butterworth low-pass filter for given specifications.	57	53	02/09/26	09/09/26	09	

Experiment no. - 01.

Ques:- Write a MATLAB code and plot DFT of a sequence without using the inbuilt functions.

Apparatus: MATLAB software.

Theory: Discrete Fourier Transform (DFT)

→ The Discrete Fourier Transform (DFT) is a mathematical technique used to convert a finite sequence of equally spaced samples of a function into a same length sequence of equally spaced samples of the discrete time Fourier transform (DTFT), which is a complex valued function of frequency. In simpler terms it transforms a signal from the time domain to the frequency domain.

• Mathematical formula:

For a discrete-time sequence $x(n)$ of length N , the N -point DFT $X(k)$ is defined by the following summation.

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j \frac{2\pi}{N} nk}$$

Here: $x(n)$ = input sequence in the time domain

$X(k)$ = DFT Sequence in the frequency domain

N = Total number of samples.

$n = \text{time index } (0 \leq n \leq N-1)$

$k = \text{frequency index } (0 \leq k \leq N-1)$

$J = \text{Imaginary unit } (\sqrt{-1})$

Program:

clc;

clear all;

close all;

$x_n = \text{input}('enter the sequence x(n)');$

$l_n = \text{length}(x_n);$

$x_k = \text{zeros}(1, l_n);$

for $k=0 : l_n-1;$

for $n=0 : l_n-1;$

$x_k(k+1) = x_k(k+1) + (x_n(n+1)) * \exp((-i) * 2 * \pi * k * n / l_n);$

end;

end;

$t=0 : l_n-1;$

subplot(2,2,1);

stem(t, x_n)

grid;

ylabel('Amplitude')

xlabel('time')

title('input sequence');

```
magnitude = abs(x, k);  
subplot(2 2 2);  
stem(t, magnitude);  
ylabel('Amplitude');  
xlabel('k');  
title('magnitude response');
```

```
phase = angle(x, k);  
subplot(2 2 3);  
stem(t, phase);  
ylabel('phase');  
xlabel('k');  
title('phase response');
```

Conclusion: The DFT of the given sequence was successfully obtained using MATLAB code without using the built-in FFT function. The magnitude and phase responses were plotted. This experiment helped to understand the basic working of DFT and frequency analysis of discrete time signal.

~~DATE~~
DATE

Experiment no. 02

Aim: Write a MATLAB code and plot IDFT of a sequence without using the inbuilt MATLAB functions.

Apparatus: MATLAB software.

Theory:

• Inverse Discrete Fourier Transform (IDFT)

→ It is a fundamental mathematical equation in digital signal processing (DSP) that converts frequency domain back into its original time representation.

when the object uses one or more of the following fast Fourier transform (FFT) algorithm depending on the complexity of a input and whether the output is in linear or bit-reversal.

Formula:

for an N -point sequence, the IDFT is defined as

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}$$

where, N = length of sequence

$x(n)$: Sequence in time domain

$X(k)$: Sequence in freq. domain

k : indices for the freq.

n = time domain, $j = (\sqrt{-1})$ Imaginary unit



Program:
clc

$x_n = \text{input}('Enter the sequence, x(n)')$

$l_n = \text{length}(x_n)$

$x_k = \text{zeros}(l, l_n)$

$i x_k = \text{zeros}(l, l_n)$

for $k=0:l_n-1$

for $n=0:l_n-1$

$x_k(k+1) = x_k(k+1) + (x_n(n+1) * \exp(-l)) * 2 * \pi * i * k * n$
 (l_n)

end

end

disp(x_k)

$t=0:l_n-1$

subplot(2 2 1)

stem(t, x_n)

grid

xlabel('time stem index')

ylabel('Amplitude')

title('input sequence')

magnitude = abs(x_k)

~~$t=0:l_n-1$~~

subplot(2 2 2)

stem($t, \text{magnitude}$)

grid

ylabel('Amplitude')



```

xlabel ('k')
title ('magnitude Response')
phase = angle (xk)
t=0 : tn-1
subplot (2 2 3)
stem (t, phase)
grid
xlabel ('k')
ylabel ('phase')
title ('phase Response')
for n=0 : tn-1
    for k=0 : tn-1
        ixk (n+1) = ixk (n+1) + (xk (k+1)) * exp (j * 2 * pi * k * n / tn)
    end
end
ixk = ixk / tn
t=0 : tn-1
subplot (2 2 4)
stem (t, xn)
grid
ylabel ('Amplitude')
xlabel ('Time Index')
title ('TDFT sequence')

```

Output:

Enter the sequence x_n : $[10 \ -2+3j \ -2 \ -2-2j]$



Conclusion: The IDFT of the given sequence was completed and plotted successfully without using the inbuilt function. The expected o/p was obtained.

~~1577126~~

Experiment no. -03.

Aim:- Write a MATLAB code & plot circular convolution of given sequence using MATLAB function.

Apparatus: MATLAB software.

Theory:

Circular Convolution is a mathematical operation applied to periodic discrete time signals where the output sequence wraps around continuously.

In digital signal processing, it is highly efficient to compute using the fast Fourier transform (FFT) because time domain convolution becomes simple element-wise multiplication in the frequency domain.

Formula-

For two N point periodic sequences $x(n)$ and $h(n)$, the circular convolution $y(n) = x(n) * h(n)$ is defined as

$$y(m) = \sum_{n=0}^{N-1} x(n) h(m-n)$$

where, $N = \text{length of sequence}$

$(m-n)$: time shift of circular.

Program :-

clc

a = input('Enter the sequence a(n) = ');

b = input('Enter the sequence b(n) = ');

n₁ = length(a);

n₂ = length(b);

N = max(n₁, n₂);

x = [a zeros(1, (N - n₁))];

for i = 1:N

k = i

for j = 1:n₂

H(i, j) = x(k) * b(j)

k = k - 1

if (k == 0)

k = N

end

end

end

y = zeros(1, N)

m = H

for j = 1:N

i = 1:n₁

y(j) = m(i, j) + y(j)

end

end



```
disp('The output sequence is y(n)='):
```

```
subplot(1,3,3)
```

```
disp(y)
```

```
stem(y)
```

```
xlabel('n')
```

```
ylabel('y(n)')
```

```
title('circular convolution')
```

```
subplot(1,3,1)
```

```
stem(a)
```

```
title('input x(n)')
```

```
xlabel('n')
```

```
ylabel('x(n)')
```

```
subplot(1,3,2)
```

```
stem(b)
```

```
title('input h(n)')
```

```
xlabel('n')
```

```
ylabel('h(n)').
```

Command WINDOW INPUT:

Enter the sequence $x(n) = [1 \ 2 \ 3 \ 1]$

Enter the sequence $h(n) = [4 \ 3 \ 2 \ 2]$

OUTPUT:

The output sequence is $y(n)$:

17 19 22 19



Conclusion: The circular convolution function is a simple & efficient MATLAB function for computing circular convolution. The stem plots clearly show the input sequences and the resulting circular convolution output.

~~A~~
5/8/26



Experiment no. -04.

Aim: Write a MATLAB code to apply FFT on a given sequence and plot the magnitude and phase response of the same.

Apparatus: MATLAB software

Theory:

Fast Fourier transform (FFT) is the fast method of calculating DFT. It will give same result but your calculation is easier or faster.

Formula-

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j2\pi kn/N}$$

$$W_N^k = e^{-j2\pi k/N} \Rightarrow X(k) = \sum_{n=0}^{N-1} x(n) W_N^{kn}$$

Twiddle factor

Twiddle factor is most important factor in FFT. It is a rotating complex no. used combined even and odd FFT results.

Program -
clc

n = input('enter the length of input:')

```
t = 0:1:n  
y = input('Enter the input sequence:');  
subplot(3,1,1)  
stem(y)  
grid  
title('input sequence')  
xlabel('time')  
ylabel('amplitude')  
disp(y)  
xn = y  
N = input('Enter the length of the FFT')  
Xk = fft(xn, N)  
magXk = abs(Xk)  
angXk = angle(Xk)  
k = 0:N-1  
subplot(3,1,2)  
stem(k, magXk)  
grid  
xlabel('k')  
ylabel('|X(k)|')  
title('magnitude response')  
subplot(3,1,3)  
stem(k, angXk)  
grid  
disp(Xk)
```

$xlabel('k')$

$ylabel('arg(x(k))')$

$title('angular response')$

COMMAND WINDOW INPUT -

Enter the input sequence: $[1 \ 2 \ 3 \ 4]$

OUTPUT -

output sequence: $[10 \ -2+2j \ -2 \ -2-2j]$.

Conclusion :-

FFT was successfully applied to the given sequence. The magnitude and phase responses were obtained, showing the frequency components of the signal efficiently.

5/8/26

Experiment no. 05

Aim: Write a MATLAB code to apply CTFFT on a given sequence and plot the magnitude and phase response of the same.

Apparatus: MATLAB software.

Theory

Inverse fast Fourier transform (CTFFT) is a mathematical technique which is used to convert signal from freq. domain to time domain. It will make calculation of IDFT fast and simple.

Formula

$$x(n) = \frac{1}{N} \sum_{k=0}^{N-1} X(k) e^{j2\pi kn/N}$$

N = length of sequence

$X(k)$ = freq. domain.

Program:-

```
clc
```

```
clear all
```

```
close all
```

```
x = input('Enter the sequence of freq. domain:')
```

```
ln = length(x)
```

```
t = 0:ln-1
```

$y = \text{ifft}(x)$
 $\text{mag} = \text{abs}(y)$
 $\text{phase} = \text{angle}(y)$
 $\text{disp}(\text{'output sequence'})$
 $\text{disp}(y)$
 $\text{subplot}(2, 2, 1)$
 $\text{stem}(x)$
 $x\text{label}(\text{'k'})$
 $y\text{label}(\text{'Amplitude'})$
 $\text{title}(\text{'Input sequence'})$
 $\text{subplot}(2, 2, 2)$
 $\text{stem}(t, \text{mag})$
 $x\text{label}(\text{'time'})$
 $y\text{label}(\text{'Amplitude'})$
 $\text{title}(\text{'magnitude Response'})$
 $\text{subplot}(2, 2, 3)$
 $\text{stem}(t, \text{phase})$
 $x\text{label}(\text{'time'})$
 $y\text{label}(\text{'Amplitude'})$
 $\text{title}(\text{'phase Response'})$

Output

Input frequency sequence of freq. domain $[10 - 2 + 2j - 2 - 2j]$

Output sequence in time domain is

 $[1 \ 2 \ 3 \ 4]$



Conclusion - In this experiment we have concluded that the IFFET and a step sequence was calculated and plotted successfully with using MATLAB function.

~~5/8/26~~



Experiment no. 06.

Ques. Write a MATLAB code to implement low pass FIR filter for given sequence.

Apparatus - MATLAB software

Theory -

Low pass filter is a kind of filter where it will allow low freq. components from the signal and it will attenuates high frequency components.

First Impulse Response filter is used.

- (i) It is always stable
- (ii) The sequence is in finite mode
- (iii) It is easy to design.

Impulse Response of low pass filter:

$$h(n) = \frac{\sin[\omega_c(n-a)]}{\pi(n-a)}$$

where, ω_c = cutoff freq.

$$a = \frac{N-1}{2}$$

N = filter length

The ideal filter has infinite sequence which is difficult for computer to implement. Therefore window method



will be used where infinite sequence is converted into finite sequence and it will create ideal response.

Rectangular window;

→ All samples are kept equal.

→ $w(n) = 1$ for all values of 'n'.

Program:

clc

$$w_c = 0.5 * \pi$$

$$N = 5$$

$$\alpha = (N-1)/2$$

$$\epsilon = 0.001$$

$$n = 0 : 1 : N-1$$

$$h_d = \sin(w_c * (n - \alpha + \epsilon)) / (\pi * (n - \alpha + \epsilon))$$

$$w_r = \text{boxcar}(N)$$

$$h(n) = h_d * w_r$$

$$w = 0 : 1 : \pi$$

$$h = \text{freqz}(h_n, 1, w)$$

$$\text{plot}(w/\pi, \text{abs}(h))$$

hold on

$$\text{plot}(w/\pi, \text{abs}(h))$$

grid

ylabel('magnitude')

xlabel('normalised frequency')

hold off.

Conclusion - The low-pass FIR filter was successfully designed and implemented in MATLAB for the given input sequence. The filter effectively allowed low-freq. components to pass while attenuating high freq. components, demonstrating the characteristics of an FIR filter.

✓
5/8/26

Experiment no. - 08

Aim:- Write a MATLAB code to implement high pass FIR filter for given sequence.

Apparatus - MATLAB software

Theory.

High pass filter is a kind of filter which will allow high freq. component to pass through & it will attenuate low freq. component.

Mathematically,

Impulse Response of HPF

$$h(n) = \begin{cases} \frac{-\sin[\omega_c (n-\alpha)]}{\pi (n-\alpha)} & , n \neq \alpha \\ 1 - \frac{\omega_c}{\pi} & , n = \alpha \end{cases}$$

Program:-

```
clc
wc = 0.5 * pi
N = 5
alpha = (N-1)/2
eps = 0.001
n = 0:1:N-1
hd = (sin(pi * (n - alpha + eps)) - sin((wc * (n - alpha + eps)) / (eps * (n - alpha + eps))));
```



```
wr = boxcar(N)
```

```
hn = fd.*wr
```

```
w = 0:.1:pi
```

```
h = freqz(hn, 1, w)
```

```
plot(w/pi, abs(h))
```

```
hold on
```

```
plot(w/pi, abs(h))
```

```
grid
```

```
ylabel('magnitude')
```

```
xlabel('normalised frequency')
```

```
hold off.
```

Conclusion :- The high pass filter was successfully implemented in MATLAB. The results confirmed that low-freq. components were attenuated while high freq. components were passed.

12/8/26



Experiment no. 08.

Aim: Write a MATLAB code to implement Band pass FIR filters for given sequence.

Apparatus - MATLAB software.

Theory-

Band pass filter allows only a specific range of frequencies to pass and it will attenuates frequencies below and above the range.

mathematically,

Band pass FIR filter,

$$h_d(n) = \begin{cases} \frac{\sin \omega C_2 (n-\alpha) - \sin \omega C_1 (n-\alpha)}{\eta (n-\alpha)} & , n \neq \alpha \\ \frac{\omega C_2 - \omega C_1}{\eta} & , n = \alpha. \end{cases}$$

Program:-

```
clc
```

```
clear all
```

```
WC1 = 0.25 * pi
```

```
WC2 = 0.75 * pi
```

```
N = 5
```

```
alpha = (N-1)/2
```

$$\epsilon ps = 0.001$$

$$n = 0 : 1 : N - 1$$

$$h_d = (\sin(\omega_c2 * (n - \alpha ph_0 + \epsilon ps)) - \sin(\omega_c1 * (n - \alpha ph_0 + \epsilon ps))) / (2 * \pi * (n - \alpha ph_0 + \epsilon ps))$$

$$w_0 = \text{boxcar}(N)$$

$$h_n = h_d * w_0$$

$$w = 0 : 0.01 : \pi$$

$$h = \text{freqz}(h_n, 1, w)$$

$$\text{plot}(w / \pi, \text{abs}(h))$$

$$\text{xlabel}('magnitude')$$

$$\text{xlabel}('normalised frequency')$$

Conclusion :- The MATLAB code successfully designs and implements a Band pass filter using rectangular window method. The filter allows frequencies between the lower cutoff freq. and upper cutoff freq. while attenuating all other frequencies.

A
26/8/26

Experiment no. - 09

Aim:- To design a Hilbert transform function using Bartlett and Hamming windows.

Apparatus: MATLAB software.

Theory-

Hilbert transform is a kind of filter which

- shifts phase of positive freq. by -90°
- shifts phase of negative freq. by $+90^\circ$
- magnitude will remain constant

Ideal impulse response of Hilbert transform.

$$h_d(n) = \frac{1 - \cos[\pi(n - \alpha)]}{\pi(n - \alpha)}$$

$$\alpha = \frac{N-1}{2}, \quad \begin{matrix} n = \alpha \\ n \neq \alpha \end{matrix}$$

Bartlett window is a kind of window which is used to make filter finite in length and reduce ripples in freq. response.

$$w_B(n) = \frac{2n}{N-1}, \quad 0 \leq n \leq \frac{N-1}{2}$$

$$w_B(n) = 2 - \frac{2n}{N-1}, \quad \frac{N-1}{2} \leq n \leq N-1$$



Program:-

clc

N=5

alpha = (N-1)/2

eps = .001

n = 0 : 1 : N-1

hd (cos(alpha + 1)) = 0

wt = bartlett(N)

hn = hd.*wt

w = 0 : .01 : pi

h = freqz(hn, 1, w)

plot(w/pi, abs(h))

hold on

wh = hamming(N)

hn = hd.*wh

w = 0 : 0.1 : pi

h = freqz(hn, 1, w)

plot(w/pi, abs(h), '-')

y label ('magnitude'); x label ('normalised frequency')

hold off

Conclusion: In this experiment, we have concluded that matlab code to apply Hilbert transform function using Bartlett and Hamming window.

09/09/26

Experiment no. - 10

Aim: To design a Butterworth low pass filter for given specification

Apparatus: MATLAB software

Theory -

Butterworth filter is used to produce smooth and ripple free frequency response.

Nyquist frequency: $f_N = f/2$

normalised stop band & band pass freq. $\Omega_p = \frac{2 \times f_p}{f}$

$$\Omega_s = \frac{2 \times f_s}{f}$$

Pre-wrapping digital freq.: $\Omega_p' = \tan\left(\frac{\pi \Omega_p}{2}\right)$

$$\Omega_s' = \tan\left(\frac{\pi \Omega_s}{2}\right)$$

order filter

$$n > \frac{\log_{10} \left[\frac{10^{\alpha_s/10} - 1}{10^{\alpha_p/10} - 1} \right]}{2 \log_{10} \left(\frac{\Omega_s'}{\Omega_p'} \right)}$$

Normalised cutoff freq.

$$\omega_n = \frac{\omega}{\omega_c} \cos^{-1} \left[\frac{\cos\left(\frac{\pi n P}{2}\right)}{\left(10^{(P/10)} - 1\right)^{1/2N}} \right]$$

Program:-

```
clc
alpha_p = 0.4
alpha_s = 30
fp = 400
fs = 800
f = 2000
om_p = 2 * fp / f
om_s = 2 * fs / f
[n, wn] = buttord(om_p, om_s, alpha_p, alpha_s)
[b, a] = butter(n, wn)
w = 0:0.01:pi
[h, om] = freqz(b, a, w, 'whole')
m = 20 * log10(abs(h))
a_n = angle(h)
subplot(2,1,1)
plot(om/pi, m)
grid
xlabel('gain in db')
```

xlabel('normalised frequency')

subplot(2,1,2)

plot(om/pi,om)

grid

ylabel('gain in radians')

xlabel('normalised frequency').

Conclusion - In this experiment, we have concluded that MATLAB code the apply Butterworth low pass filter for a specifications and was calculated & plotted successfully using MATLAB function.

A
09/09/26